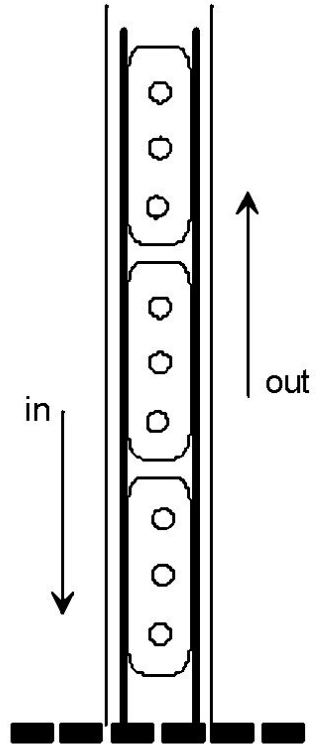


CENG 202. Data Structures

Lecture 10. Stack ADT



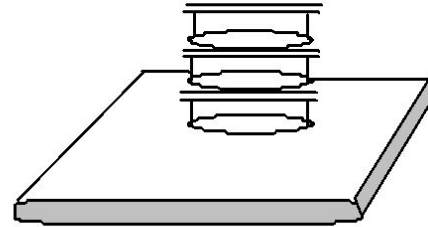
Concept of Stack Abstract Data Type



Trains in a Railyard



Goods in a cargo

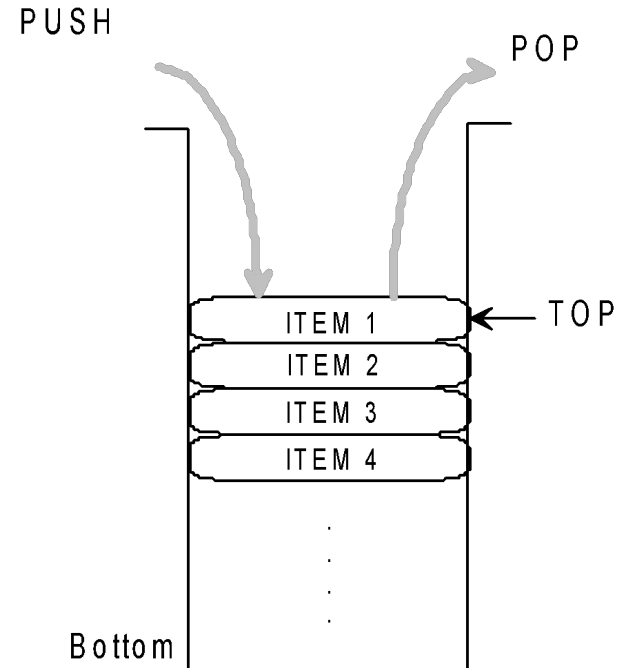


Plates on a tray

Definition of Stack

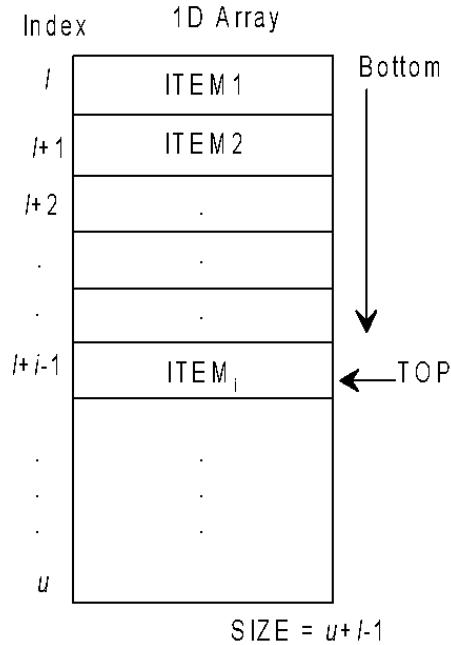
A stack is an ordered collection of homogeneous data element where the insertion and deletion operations take place at one end only.

Stack follows **LIFO (Last In First Out)** property

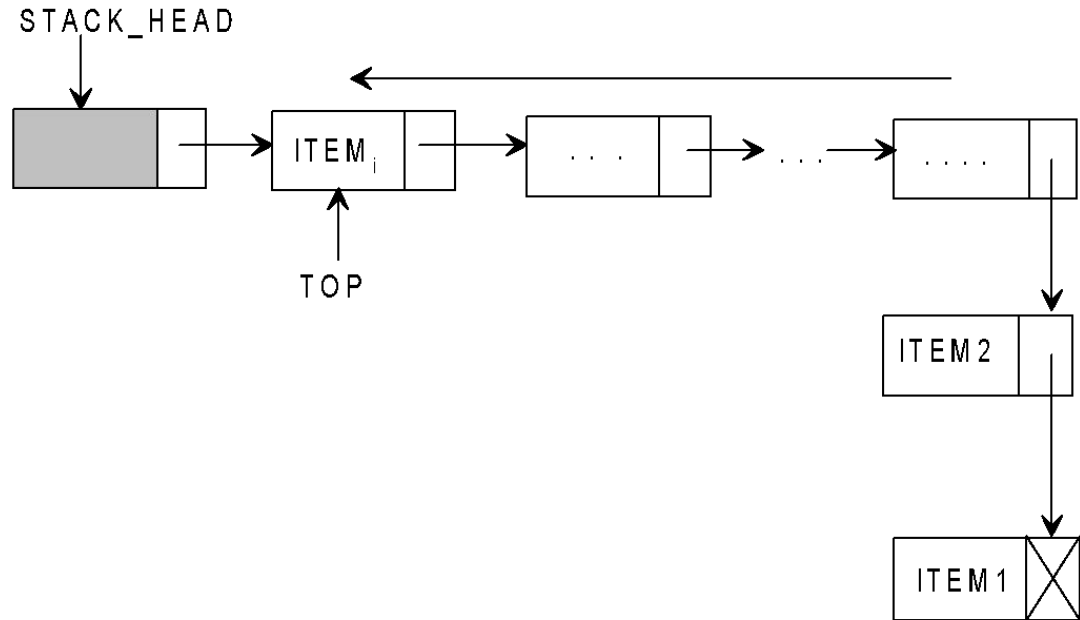


Memory representations for Stack

Array Representation



Linked-list Representation



Operations on Stack

Push

To **insert** an item into the stack

Pop

To **remove** an item from a stack

Status

To know the present state of a stack

- if stack **is empty**,
- **size** of the stack,
- the element **at top**

Example

Method	Return Value	Stack Contents
push(5)	—	(5)
push(3)	—	(5, 3)
size()	2	(5, 3)
pop()	3	(5)
isEmpty()	false	(5)
pop()	5	()
isEmpty()	true	()
pop()	null	()
push(7)	—	(7)
push(9)	—	(7, 9)
top()	9	(7, 9)
push(4)	—	(7, 9, 4)
size()	3	(7, 9, 4)
pop()	4	(7, 9)
push(6)	—	(7, 9, 6)
push(8)	—	(7, 9, 6, 8)
pop()	8	(7, 9, 6)

Operation: Push with array representation

We have assumed that the **array index varies from 1 to SIZE** and TOP points the location of the current top-most item in the stack. The following algorithm defines the insertion of an item into a stack represented using an array A.

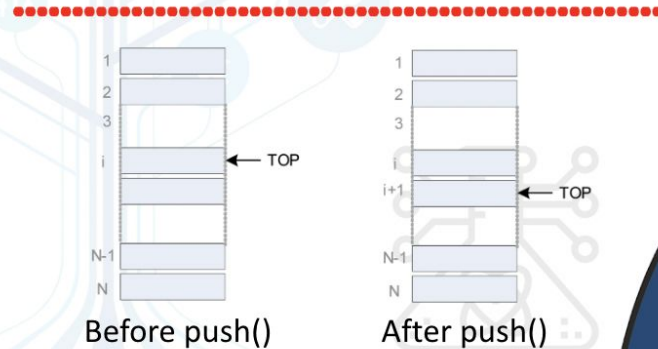
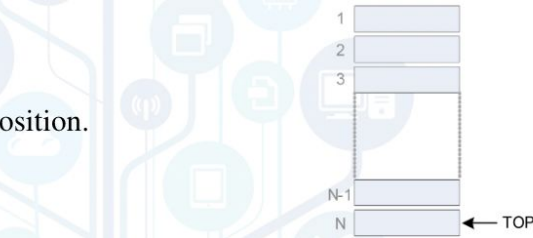
Input: The new item ITEM to be pushed onto it.

Output: A stack with a newly pushed ITEM at the TOP position.

Data structure: An array A with TOP as the pointer.

Steps:

1. **If** $TOP \geq SIZE$ **then**
2. **Print** "Stack is full"
3. **Else**
4. $TOP = TOP + 1$
5. $A[TOP] = ITEM$
6. **EndIf**
7. **Stop**



Operation: Pop with array representation

Input: A stack with elements.

Output: Removes an ITEM from the top of the stack if it is not empty.

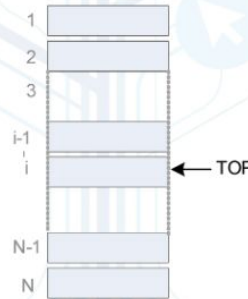
Data structure: An array A with TOP as the pointer.

Steps:

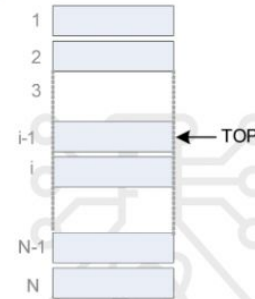
1. **If** $TOP < 1$ **then**
2. **Print** "Stack is empty"
3. **Else**
4. $ITEM = A[TOP]$
5. $TOP = TOP - 1$
6. **EndIf**
7. **Stop**



Pop operation is failed



Before Pop()



After Pop()

Operation: Status with array representation

The following algorithm tests the various states of a stack such as whether it is full or empty, how many items are right now in it, and read the current element at the top without removing it, etc.

Steps:

1. **If** TOP < 1 **then**
2. **Print** "Stack is empty"
3. **Else**
4. **If** (TOP ≥ SIZE) **then**
5. **Print** "Stack is full"
6. **Else**
7. **Print** "The element at TOP is", A[TOP]
8. free = (SIZE – TOP)/SIZE * 100
9. **Print** "Percentage of free stack is", free
10. **EndIf**
11. **EndIf**
12. **Stop**

Performance and Limitations in Array-based Stack ADT

Performance

- Let n be the number of elements in the stack
- The space used is $O(n)$
- Each operation runs in time $O(1)$ (constant time)

Limitations

- The maximum size of the stack must be defined a priori and cannot be changed
- Trying to push a new element into a full stack causes an implementation-specific exception

Stack with Linked List

Operation: Push with linked list

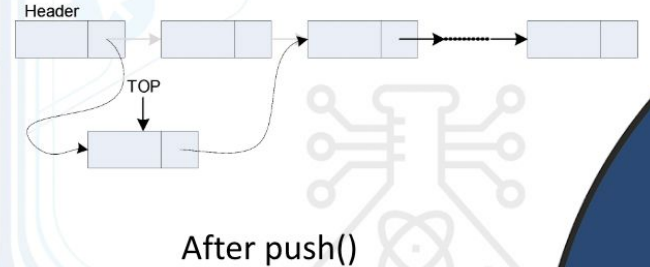
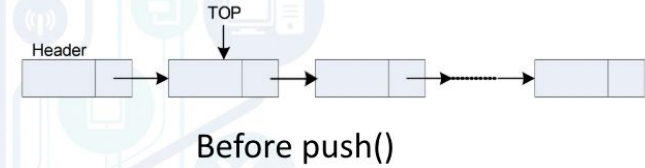
Input: ITEM is the item to be inserted.

Output: A single linked list with a newly inserted node with data content ITEM.

Data structure: A single linked list structure whose pointer to the header is known from STACK_H and TOP is the pointer to the first node.

Steps:

1. new = **GetNode**(NODE)
/* Insert at front */
2. new → DATA = ITEM
3. new → LINK = TOP
4. TOP = new
5. STACK_H → LINK = TOP
6. **Stop**



Operation: Pop with linked list

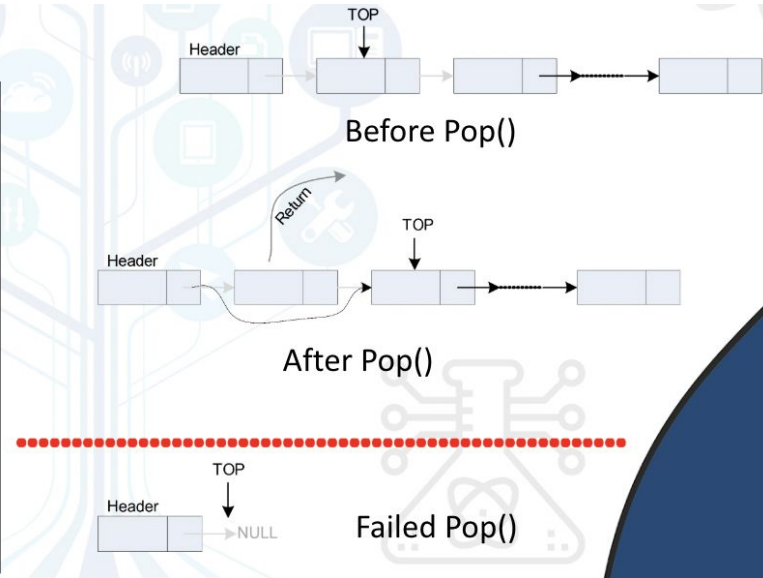
Input: A stack with elements.

Output: The removed item is stored in ITEM.

Data structure: A single linked list structure whose pointer to the header is known from STACK_H and TOP is the pointer to the first node.

Steps:

1. **If** TOP = NULL
2. **Print** "Stack is empty"
3. **Exit**
4. **Else**
5. ptr = TOP → LINK
6. ITEM = TOP → DATA
7. STACK_H → LINK = ptr
8. TOP = ptr
9. **EndIf**
10. **Stop**



Operation: Status with linked list

Input: A stack with elements.

Output: Status information such as its state (empty or full), number of items, item at the TOP.

Data structure: A single linked list structure whose pointer to the header is known from STACK_H and TOP is the pointer to the first node.

Steps:

1. ptr = STACK_H → LINK
2. **If** (ptr = NULL) **then**
3. **Print** "Stack is empty"
4. **Else**
5. nodeCount = 0
6. **While** (ptr ≠ NULL) **do**
7. nodeCount = nodeCount + 1
8. ptr = ptr → LINK
9. **EndWhile**
10. **Print** "Top item:", TOP → DATA,
 "Size = ", nodeCount
11. **EndIf**
12. **Stop**

Common Stack Applications

Page-visited history in a Web browser

Evaluation of arithmetic expression

- Conversion an expression to postfix notation
- Evaluation for the value
- Machine code generation

Execution of recursive call of a function

- Factorial calculation
- Tower of Hanoi
- Quick sort
- Merge sort

Scope rule implementation

- Static and dynamic scope rule management
- Activation record management

Memory management

- Run-time memory management
- Code generation
- Syntax parsing

Undo sequence in a text editor

Parenthesis Matching

HTML Tag Matching

Application: Parentheses Matching

Each “(”, “{”, or “[” must be paired with a matching “)”, “}”, or “]”

- correct: () (()) { ([()]) }
- correct: ((() (()) { ([()]) }
- incorrect:) (()) { ([()]) }
- incorrect: { ([]) }
- incorrect: (

Application: Parentheses Matching

Algorithm for matching parentheses

1. Each time we encounter an **opening symbol**, we **push** that symbol onto the stack.
2. Each time we encounter a **closing symbol**, we **pop** a symbol from the stack (assuming it is not empty) and check that these two symbols form a valid pair.
3. If we reach the end of the expression and the stack is empty, then the original expression was properly matched.
4. Otherwise, there must be an opening delimiter on the stack without a matching symbol.

If the length of the original expression is n , the algorithm will make at most n calls to push and n calls to pop.

Application: HTML Tag Matching

For fully-correct HTML, each <name> should pair with a matching </name>

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

The Little Boat

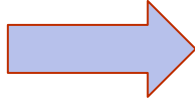
The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

Application: Evaluating Postfix Arithmetic Expressions

- The operator is placed before its two operands.
- Here no parentheses are required, i.e.,

A+B



AB+

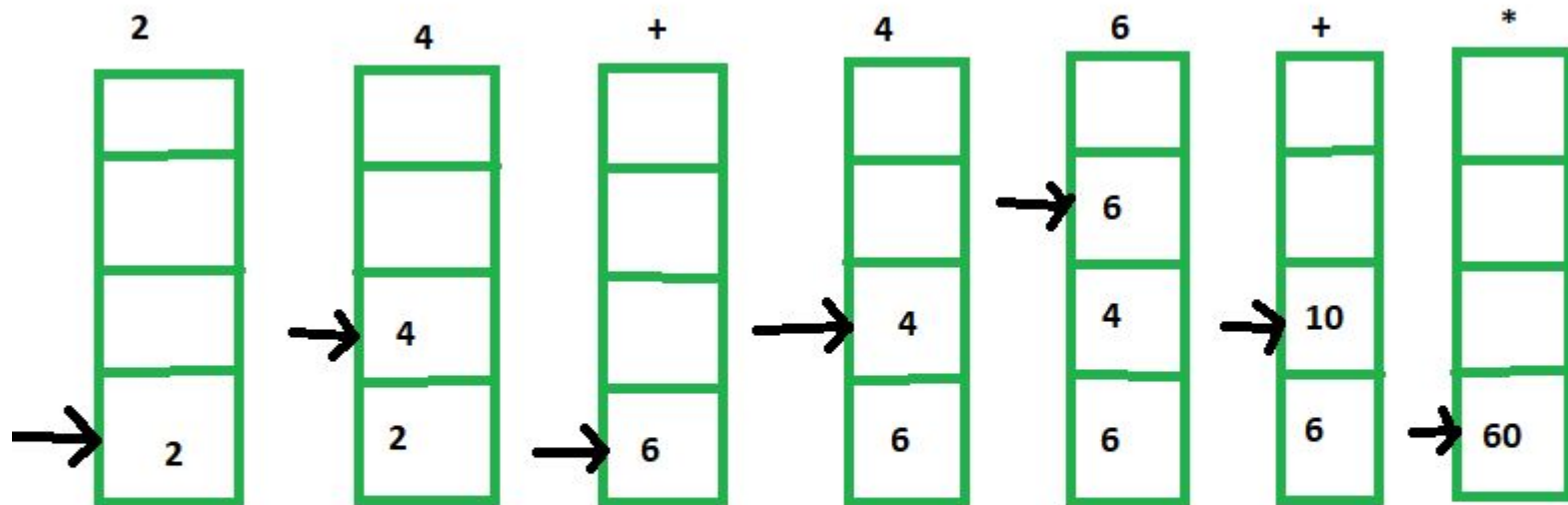
(2+4) * (4+6)

2 4 + 4 6 + *

Algorithm for Evaluating Postfix Expressions

1. Push the operands into the stack in the order they appear.
2. When any operator encounters then pop two topmost operands for executing the operation.
3. After execution push the result obtained into the stack.
4. After the complete execution of expression, the final result remains on the top of the stack.

Example: 2 4 + 4 6 + *



Stack operations to evaluate $(2+4)*(4+6)$

Application: Infix arithmetic expression evaluation

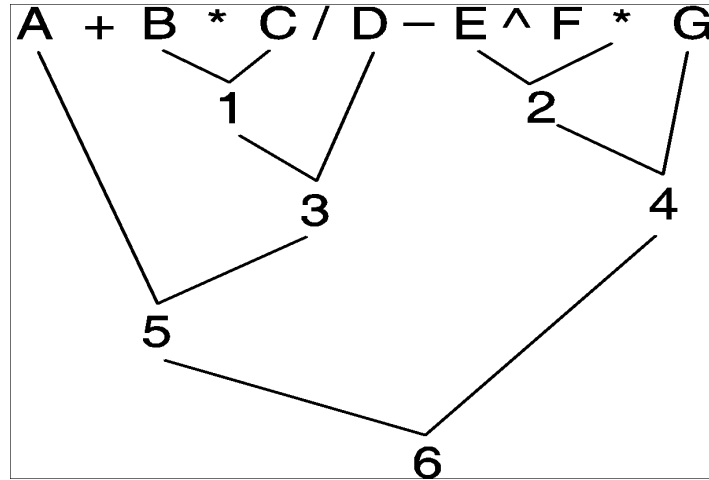
$A + B * C / D - E \wedge F * G$

Precedence and associativity of operators

<i>Operators</i>	<i>Precedence</i>	<i>Associativity</i>
$-$ (unary), $+$ (unary), NOT	6	$-$
$\wedge$ (exponentiation)	6	Right to left
$*$ (multiplication), $/$ (division)	5	Left to right
$+$ (addition), $-$ (subtraction)	4	Left to right
$<$, $<=$, $+$, $<>$, $>=$	3	Left to right
AND	2	Left to right
OR, XOR	1	Left to right

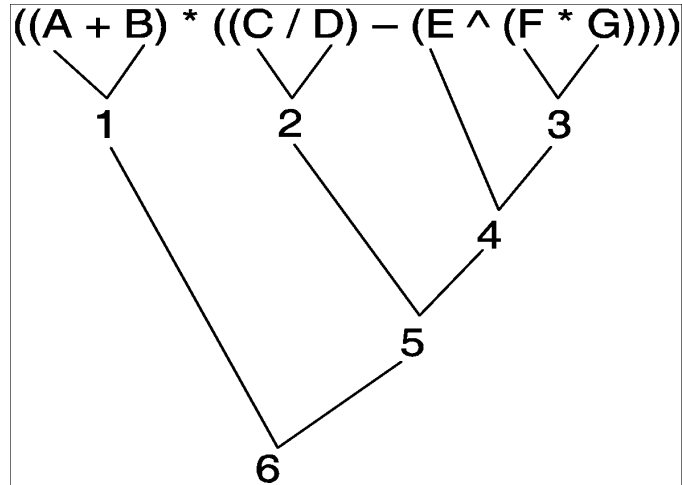
Application: Infix arithmetic expression evaluation

$A + B * C / D - E \wedge F * G$



Application: Infix arithmetic expression in FORCED parenthesized form

$((A + B) * ((C/D) - (E \wedge (F * G))))$



Application: Infix arithmetic expression evaluation

$$14 - 3 * 2 + 7 = (14 - (3 * 2)) + 7$$

Operator precedence

* has precedence over +/−

Associativity

operators of the same precedence group
evaluated from left to right

Example: $(x - y) + z$ rather than $x - (y + z)$

Idea: push each operator on the stack, but first pop and perform higher and *equal* precedence operations.

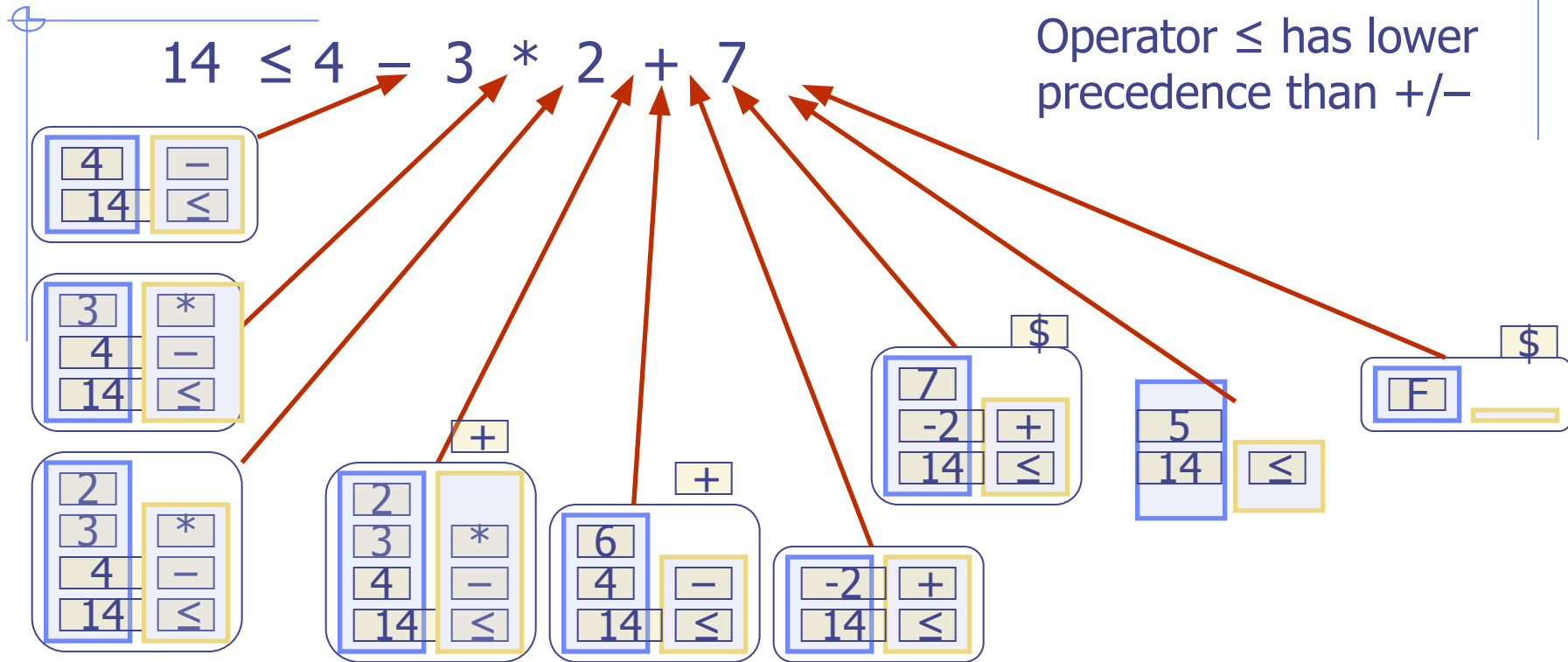
Algorithm for Evaluating Infix Expressions

An infix expression is evaluated using two stacks, one for operator and another for operands.

Algorithm:

- If symbol is an operand, push it in operand's stack.
- Initialize operator stack with a dummy operator with the least precedence (say #).
- If the symbol is an operator, then do the following:
 - Pop an operator from the operator stack.
 - Check its precedence with the symbol.
 - If the precedence of the symbol is higher, then simply push the popped operator and then the symbol.
 - Else if the precedence of symbol is lower (or equal to) than the popped operator, pop two operands, (opnd2 and opnd1) and perform operation specified by popped operator.
 - Push the result in operand's stack.
 - Repeat above steps until the symbol becomes less than the popped operator.
- After the string ends, start popping an operator from the operator stack and two operands from operand stack.
- Repeat step (d) until dummy operator (#) is found.
- Pop the value from the operand stack and return it.

Example 1



Example - 2

Example: $4 + 5 * 3 - 2$

Symbol	opndstk	optrstk	popped optr	opnd1	opnd2	value
4	4	#				
+	4	#, +				
5	4, 5	#, +				
*	4, 5	#, +, *				
3	4, 5, 3	#, +, *				
-	4	#, +	*	5	3	15
	4, 15	#	+	4	15	19
	19	#, -				
2	19, 2	#, -	-	19	2	17
	17	#				

Result